

Documento Técnico: Paquete D - Plataforma, Integración Transversal y Seguridad

Responsable: Anthonny (Persona 4 — Integrador Técnico) **Rama de trabajo:** `feature/paquete-d-platform`

1. Responsabilidad y Alcance

El Paquete D constituye la **plataforma transversal** de la Plataforma OSINT Ecuador: el punto de entrada único al sistema (API Gateway), el bus de eventos que desacopla a los Paquetes A, B y C (RabbitMQ), el servicio que cierra el ciclo de vida del reporte notificando al ciudadano y al equipo de compliance (Notificaciones), la observabilidad centralizada (ELK) y el `docker-compose.yml` raíz que integra los 4 paquetes en un solo despliegue reproducible. Además, consolida la documentación arquitectónica global (C4, ADRs, atributos de calidad) y las plantillas de CI/CD.

A diferencia de los Paquetes A, B y C — que construyen una aplicación de negocio — el Paquete D no tiene lógica de dominio propia: su responsabilidad es que las piezas de los demás equipos se comuniquen de forma segura, resiliente y observable.

2. Arquitectura y Stack Tecnológico

El componente está segmentado en cinco piezas:

API Gateway (`app4-platform/gateway-config/`, `gateway.Dockerfile`):

- **Servidor:** NGINX 1.25 (Alpine).
- **TLS:** certificado autofirmado generado en tiempo de build (`openssl req -x509`), servido en el puerto `443` con redirección forzada desde `80`.
- **Rate Limiting:** `limit_req_zone` — 20 peticiones/minuto por IP, con *burst* de 5.
- **"WAF ligero":** bloqueo de patrones comunes de SQLi/XSS/path traversal en la query string vía `map + if`. No sustituye un WAF de producción (ModSecurity/OWASP CRS); es una mitigación razonable para el alcance académico del proyecto.

Bus de Eventos (`app4-platform/rabbitmq-config/`):

- **Broker:** RabbitMQ 3 (`rabbitmq:3-management-alpine`).
- **Topología declarativa:** `definitions.json` cargado al arrancar (`management.load_definitions`), declara los 3 exchanges *topic* (`solicitud.exchange`, `osint.exchange`, `reporte.exchange`), el exchange `osint.dlx` (direct) y las colas `solicitud.dlq` / `completado.dlq`.
- **Dead-lettering sin tocar código ajeno:** en vez de redeclarar las colas de App1/App2/App3 (lo que rompería si sus argumentos no coinciden byte a byte), se usan **polícies** de RabbitMQ que aplican `dead-letter-exchange` por patrón de nombre de cola.

Servicio de Notificaciones (`app4-platform/notifications/`):

- **Framework:** Java 17 + Spring Boot 3.2 (`spring-boot-starter-websocket`, `spring-boot-starter-amqp`).
- **WebSockets:** STOMP sobre SockJS, endpoint `/ws`, tópicos `/topic/reports/{requestId}` y `/topic/compliance/alerts`.

- **No envía correo.** El correo real lo dispara el frontend de App2 vía EmailJS (ver ADR-005) — este servicio solo reenvía el push por WebSocket, para no depender de credenciales SMTP compartidas por todo el equipo.
- Sigue el mismo patrón de `RabbitMQConfig` que ya usan App2 y App3, para que el código sea familiar al resto del equipo.

Observabilidad (`app4-platform/elk-config/`):

- **Logstash 8.8.2:** input `gelf` (UDP 12201, driver nativo de Docker), output a Elasticsearch con índice `logs-%{+YYYY.MM.dd}`.
- **Kibana 8.8.2.**
- Reutiliza el `app3-elasticsearch` ya existente (namespace de índice separado del de Compliance) en vez de levantar un segundo clúster, para ahorrar RAM.
- Activado únicamente bajo el **Compose profile** `elk`, apagado por defecto.

Orquestación (`docker-compose.yml` raíz):

- Unifica los contenedores de los 4 paquetes respetando el aislamiento de red original: `net-app1`, `net-app2`, `net-app3` nunca se comunican entre sí de forma directa; toda comunicación inter-app pasa por `net-transit` (RabbitMQ). Las únicas excepciones deliberadas son el Gateway (necesita alcanzar los backends de App2/App3 para enrutar) y la infraestructura ya compartida (`app4-localstack`, `app3-elasticsearch`).

3. Contratos de Integración Basados en Eventos

El servicio de Notificaciones consume dos eventos publicados por otros paquetes:

3.1. Evento `reporte.listo` (publicado por App2)

- **Exchange / Routing Key:** `reporte.exchange` / `reporte.listo`
- **Cola propia creada por App4:** `notificaciones.reporte.listo.queue`
- **Acción:** transmite un push por WebSocket al tópico `/topic/reports/{requestId}`. El correo real al ciudadano con el enlace del PDF lo dispara por separado el frontend de App2 (vía EmailJS) al detectar el estado `COMPLETED`, no este servicio.
- **Estructura del Payload (JSON):** `json { "requestId": "UUID", "pdfUrl": "https://s3.amazonaws.com/osint-bucket/reports/...pdf", "email": "ciudadano@mail.com", "status": "COMPLETED" }`

3.2. Evento `alerta.compliance` (publicado por App3)

- **Exchange / Routing Key:** `osint.exchange` / `alerta.compliance`
- **Cola propia creada por App4:** `notificaciones.alerta.compliance.queue`
- **Acción:** transmite un push de alta prioridad por WebSocket al tópico `/topic/compliance/alerts` (sin email, según el diseño del sistema).
- **Estructura del Payload (JSON):** `json { "alertId": "UUID", "requestId": "UUID", "targetId": "cedula de 10 dígitos", "fullName": "NOMBRE COMPLETO", "riskLevel": "HIGH", "matchReason":`

"Coincidencia 89% con lista de PEPs/Sanciones", "timestamp": 1782813410000 }

4. Especificación de Rutas del API Gateway

El Gateway (<https://localhost>) enruta hacia los backends internos sin exponer sus puertos directamente:

Ruta pública	Destino	Notas
POST/GET /api/v1/reports/**	app2-backend:8080	Rate limit 20 req/min, CORS habilitado
GET/POST /api/v1/compliance/**	app3-backend:8081	Rate limit 20 req/min, CORS habilitado
/ws/**	app4-notifications:8082	Proxy con Upgrade/Connection para WebSocket
/docs/portal/v3/api-docs	app2-backend:8080/v3/api-docs	Spec OpenAPI cruda del Portal (para importar a SwaggerHub)
/docs/compliance/v3/api-docs	app3-backend:8081/v3/api-docs	Spec OpenAPI cruda de Compliance

Los frontends (React) siguen siendo accesibles de forma directa en sus puertos publicados (5173, 5174) tal como documenta [guia-integracion-osint.md](#); el Gateway es un punto de entrada adicional centralizado, no un requisito para que el sistema funcione.

5. Procedimiento de Despliegue Local

5.1. Levantar el ecosistema completo (los 4 paquetes)

Desde la raíz del repositorio:

```
docker-compose up -d
```

Esto construye y levanta frontends, backends, workers, bases de datos, RabbitMQ, Gateway y Notificaciones, respetando el aislamiento de red (ver [docs/infrastructure/topologia-red-docker.drawio](#)).

5.2. Envío de correo real

No requiere configuración adicional: el correo lo dispara el navegador del ciudadano vía **EmailJS** cuando el Portal detecta que el reporte quedó **COMPLETED** (ver [app2-portal/frontend/.env](#) y ADR-005). No hay ningún [.env](#) de backend que completar para esto — `docker-compose up` funciona igual para todo el equipo sin configuración de correo extra.

5.3. Levantar solo la plataforma transversal (Paquete D)

```
docker-compose up -d app4-rabbitmq app4-localstack app4-notifications app4-gateway
```

5.4. Activar observabilidad (ELK, opcional)

```
docker-compose --profile elk up -d
```

Kibana queda disponible en <http://localhost:5601>.

5.5. Endpoints útiles

- API Gateway (HTTPS, certificado autofirmado): <https://localhost>
- RabbitMQ Management UI: <http://localhost:15672> (guest / guest)
- WebSocket de Notificaciones: <wss://localhost/ws>

5.6. Build local del servicio de Notificaciones (sin Docker)

```
cd app4-platform/notificationsmvn -B verify
```

Requiere JDK 17 (Eclipse Temurin/Corretto recomendado — igual que App2/App3, Lombok aún no es totalmente compatible con JDK 21+).

6. Pruebas de Integración y Entorno Mock

Para probar el servicio de Notificaciones sin depender de que App2/App3 estén corriendo:

1. Abre la consola de RabbitMQ (localhost:15672) → **Queues** → [notificaciones.reporte.listo.queue](#) → **Publish message**, con el payload de la sección 3.1.
2. Conecta un cliente STOMP/SockJS a [/ws](#) y suscríbete a [/topic/reports/{requestId}](#) (usa el mismo [requestId](#) del payload) para confirmar el push en tiempo real.
3. Para probar el envío real de correo (independiente de RabbitMQ/Notificaciones), usa el flujo completo del Portal Ciudadano o llama directo a <https://api.emailjs.com/api/v1.0/email/send> con los identificadores de [app2-portal/frontend/.env](#).
4. Repite el flujo del punto 1 publicando en [notificaciones.alerta.compliance.queue](#) (exchange [osint.exchange](#), routing key [alerta.compliance](#)) y suscribiéndote a [/topic/compliance/alerts](#).
5. Para probar el DLQ: publica manualmente un mensaje malformado (JSON inválido) en [solicitud.osint.queue](#) y confirma en la consola de RabbitMQ que, tras los reintentos configurados, termina en [solicitud.dlq](#).

7. CI/CD

Se agregaron 4 plantillas de GitHub Actions en [.github/workflows/](#), una por paquete, disparadas solo cuando cambian los archivos de su carpeta: * [app1-worker-ci.yml](#): build de imágenes del Worker y la Lambda PDF. * [app2-portal-ci.yml](#): `mvn verify` + tests/build de frontend (Vitest). * [app3-compliance-ci.yml](#): `mvn verify` + build de frontend. * [app4-platform-ci.yml](#): `mvn verify` de Notificaciones, build + `nginx -t` del Gateway, y `docker compose config` del compose raíz.

8. Documentación Generada

- [docs/c4-diagrams/c4-contexto-sistema.drawio](#) y [c4-contenedores-sistema.drawio](#) — C4 Nivel 1 y 2 del sistema completo.

- [docs/infrastructure/topologia-red-docker.drawio](#) — diagrama de despliegue con las 4 redes Docker.
- [docs/architecture-decisions/ADR-001](#) a [ADR-004](#) — decisiones de arquitectura (EDA, stack Java, base de datos por servicio, PDF serverless).
- [docs/architecture-decisions/ADR-005](#) — envío de correo real desde el frontend (EmailJS) en vez de SMTP compartido en el backend.
- [docs/quality-attributes/seguridad_redundancia.md](#) — justificación de WAF/TLS/JWT/redundancia, cierra el punto pendiente de [docs/quality-attributes/README.md](#).

9. Observaciones para el Equipo (no corregidas por no ser parte de este paquete)

- **App1 (Worker):** `app1-worker/worker/app.py` enlaza su cola `solicitud.osint.queue` al exchange `osint.exchange`, pero App2 publica en `solicitud.exchange`. No rompe el flujo (RabbitMQ simplemente agrega un binding extra inofensivo a la misma cola), pero conviene corregirlo por claridad.
- **App2 (Portal):** para que el DLQ configurado en este paquete sea efectivo en `osint.completado.queue`, `app2-portal/backend/src/main/resources/application.yml` debería agregar el mismo bloque `spring.rabbitmq.listener.simple.retry` que ya usa App3 (3 intentos, luego rechaza sin reencolar).

10. Glosario Rápido

- **DLX / DLQ (Dead Letter Exchange / Queue):** exchange y cola destino donde RabbitMQ reenvía automáticamente un mensaje que fue rechazado (o agotó sus reintentos), para aislarlo sin perderlo.
- **Policy (RabbitMQ):** regla que aplica configuración adicional (como un DLX) a las colas cuyo nombre coincide con un patrón, sin necesidad de que el cliente que las declara conozca esa configuración.
- **STOMP / SockJS:** protocolo de mensajería sobre WebSocket (STOMP) con una capa de compatibilidad (SockJS) para navegadores o proxies que no soportan WebSocket nativo.
- **Rate Limiting:** control que limita cuántas peticiones puede hacer un mismo cliente (aquí, por IP) en una ventana de tiempo, para proteger el sistema de picos de tráfico.
- **Compose profile:** etiqueta opcional en `docker-compose.yml` que permite que un grupo de servicios (aquí, ELK) solo se levante si se solicita explícitamente (`--profile elk`), sin afectar el `docker-compose up -d` por defecto.